

ECE 1387 Lecture #8

Nov 7, 2025

- Partitioning by branch & bound
- A3 posted
- Ex 1 due today
- Quiz 1 today

Bipartitioning (break into 2 pieces, as opposed to k-way)

Given:

Netlist of blocks, all equal size.

→ break into k pieces

↳ represent as a graph

$G(V, E)$

$v_1, v_2 \dots v_n$: cells in design

$e_1, e_2 \dots e_n$ $e_i = (v_i, v_j) \in E$

Find:

- bipartition of nodes
- approximately equal # of nodes in each partition
- minimize # edges (wires) between partitions

(minimize cut
minimize crossing count)

Find V_1, V_2

Such that

$$V_1 \subset V, \quad V_2 \subset V$$

$$V_1 \cup V_2 = V, \quad V_1 \cap V_2 = \emptyset$$

$$||V_1| - |V_2|| < \epsilon \text{ imbalance factor}$$

Want to minimize $e = (v_i, v_j)$

$$|E_{\text{ext}}| = |\{e \in E \mid v_i \in V_1, v_j \in V_2\}|$$

↑
external
edges

crossing count

This is simple version, real problems include:

- ① unequal block sizes
- ② community constraints
- ③ different block types
- ④ fixed blocks
- ⑤ some uncuttable edges
- ⑥ K-way
- ⑦ special handling for high fanout nets -
- ⑧ duplication
- ⑨ performance constraints

Complexity of bipartitioning:

if $|V| = n$,

How many partitionings?

$\binom{n}{n/2}$ for equal balance

If balance is not mandatory?
 2^n

Branch & bound (B&B)

general technique, guarantees
optimal solution.

— can be applied to many
problems.

Step 1 : B&B requires a tree to represent all possible solutions called decision tree

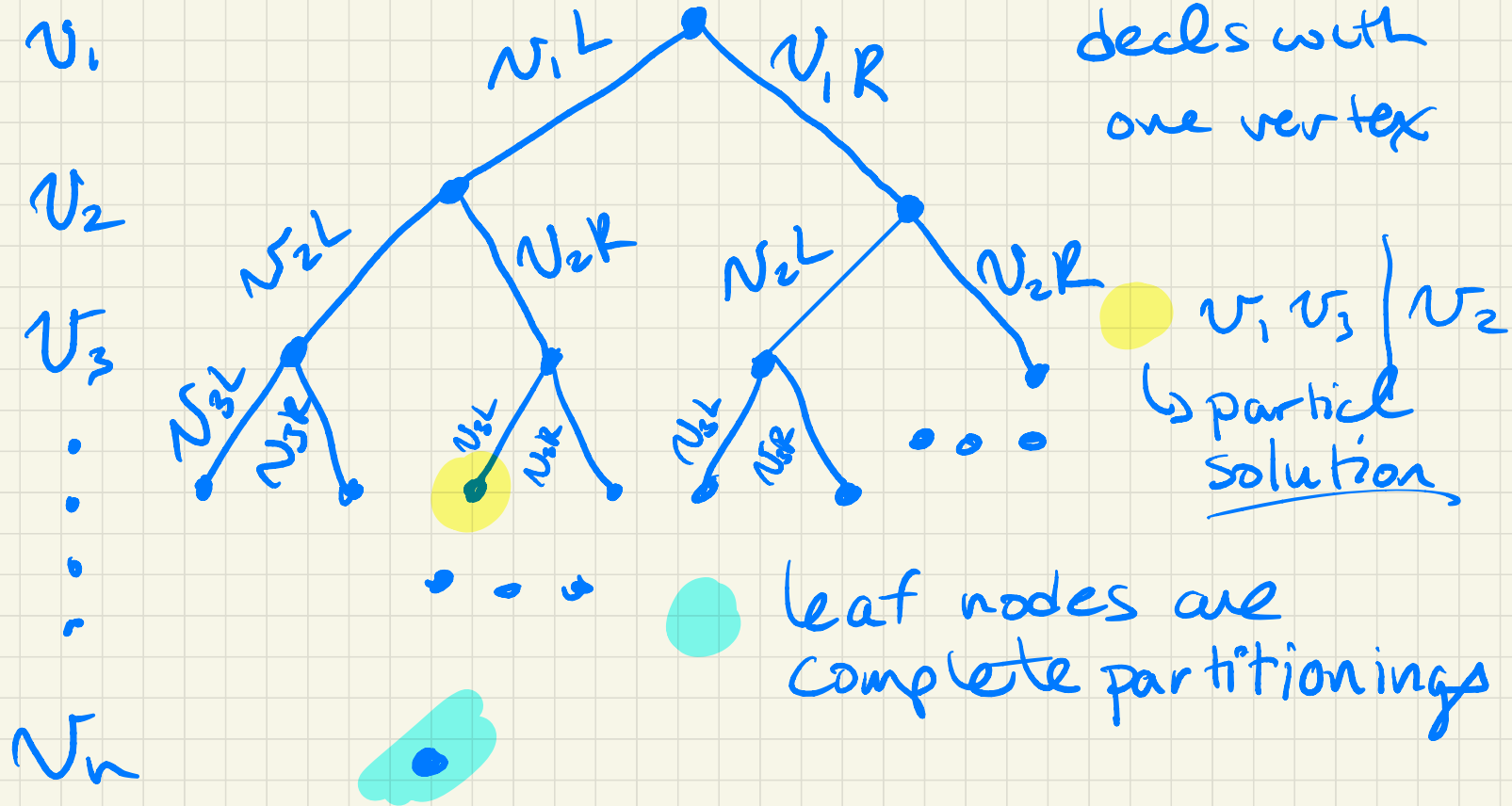
- won't necessarily build whole tree $\Rightarrow$ we will prune it as we go

Set a vert $v_1, v_2, \dots, v_n$
want to know if

$v_i \in V_1$ or $v_i \in V_2$
left right

one possible decision tree

each level
deals with
one vertex



for partial solutions, every node below contains that partial solution.

- Naive way: create entire tree
- B&B is a clever way to avoid looking at many solutions that are provably not optimal

Step 2: Initial Solⁿ

Any initial solution (best one we can get quickly)

Let cost of initial solution

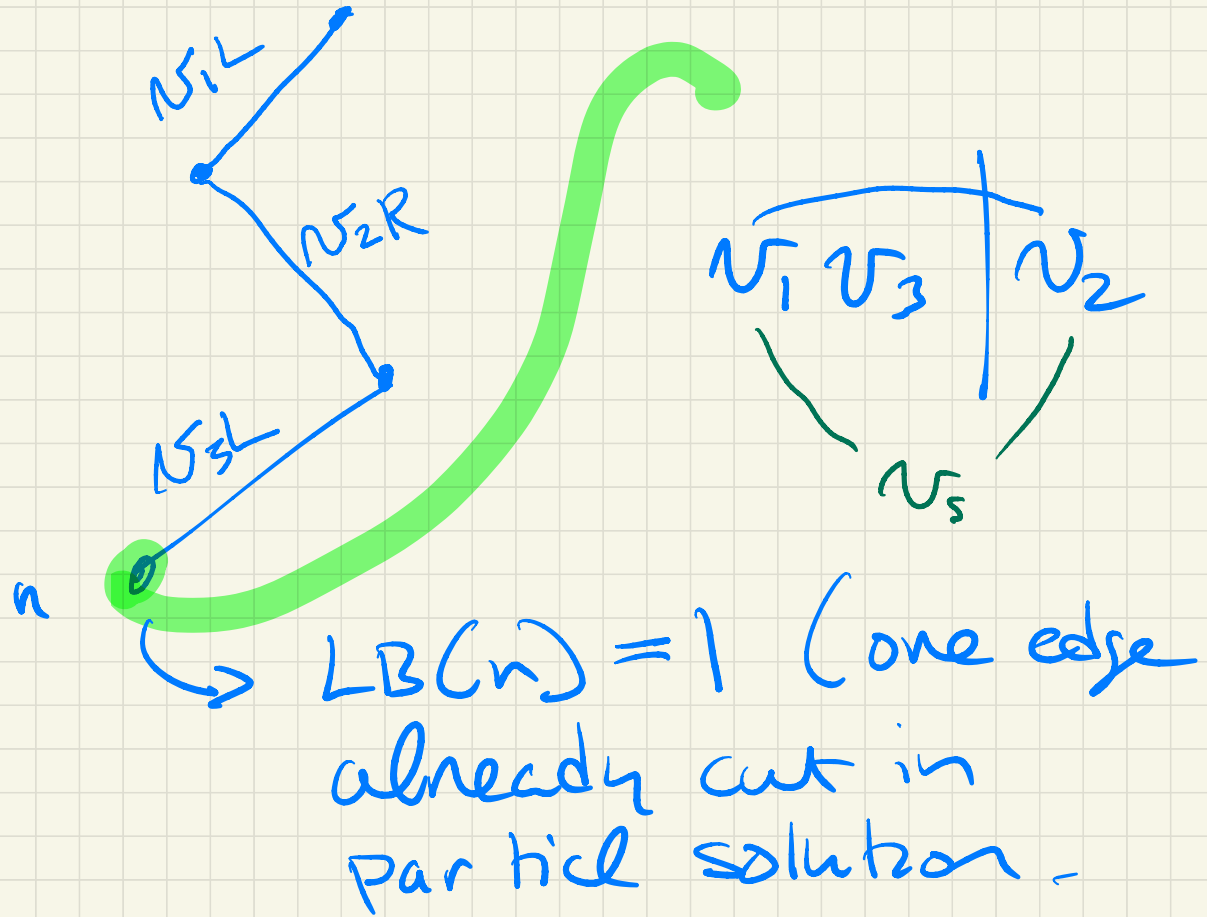
(crossing count) $\equiv$ Best

How?

- walk circuit graph to decide initial partition
- randomly ($\frac{1}{2}$ left, $\frac{1}{2}$ right)
- other heuristic (greedy swaps)

Step 3: Bounding Function

- Create a bounding function that can be applied to any node on tree, $LB(n)$
- $LB(n)$: lower bound on the cost of all possible solutions underneath node n .
($\Rightarrow$ no solⁿ under n will cost less than $LB(n)$)



- the higher the better
- requires some cleverness/
innovation
- If not a true lower bound,
optimality is not guaranteed.

Step 4: Traverse Tree

- from root down
- create tree while traversing
(not all up front)

for each node n visited

if n is a leaf and $\text{cost}(n) < \text{Best}$
replace best

if balance violated (n)

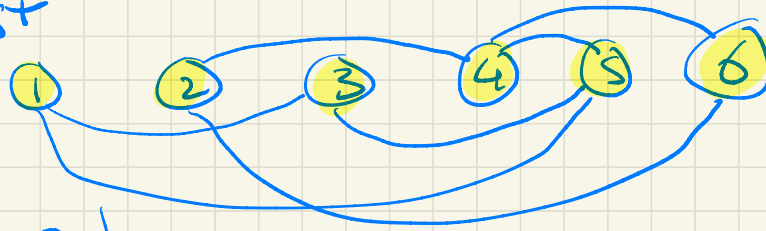
prune // don't traverse below

if $LB(n) \geq \text{Best}$
prune // don't traverse below

— Avoid enumerating solutions
provably bad (not optimal)

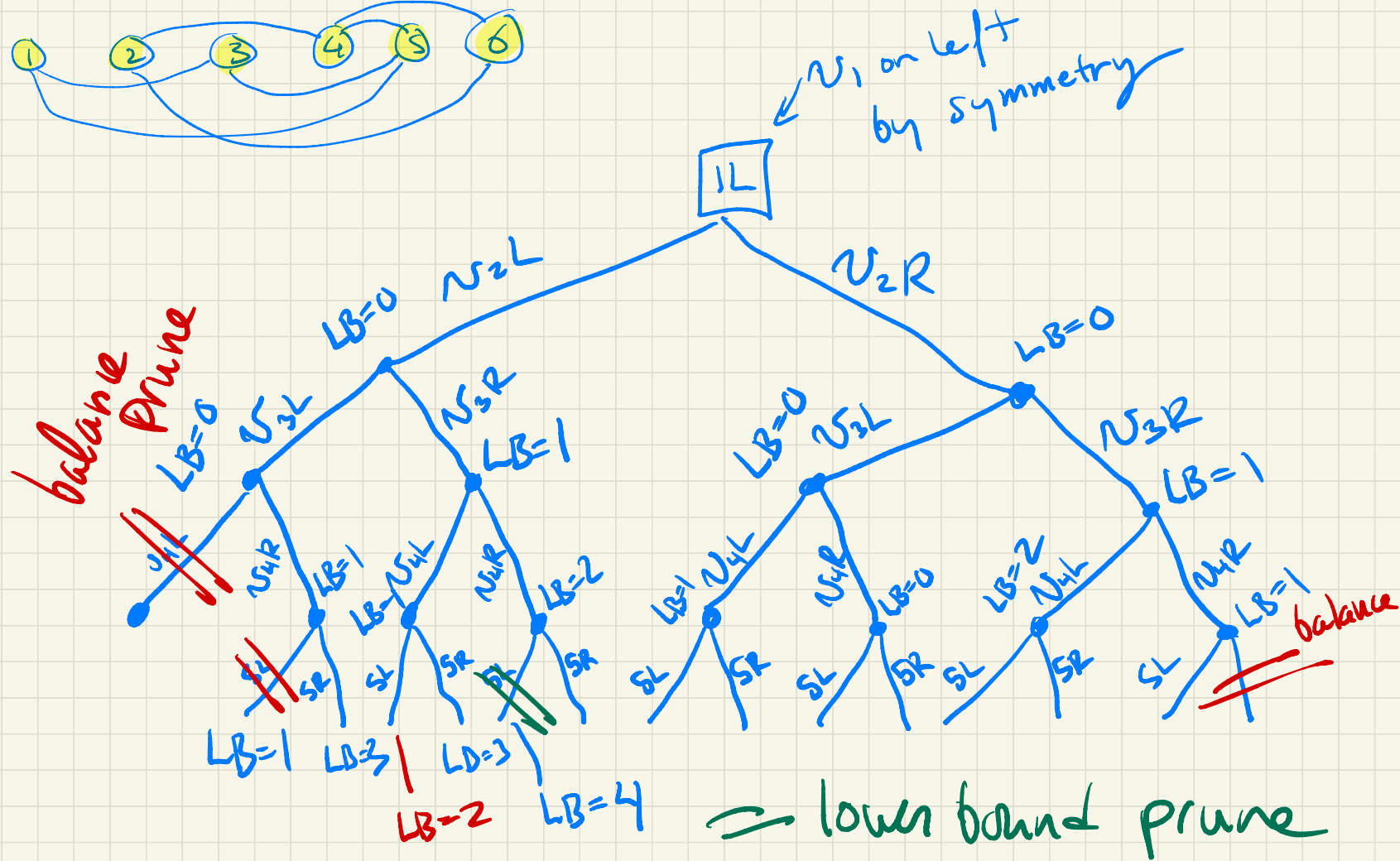
Step 5: After entire tree is
pruned or traversed, **Best**
is optimal

Example netlist

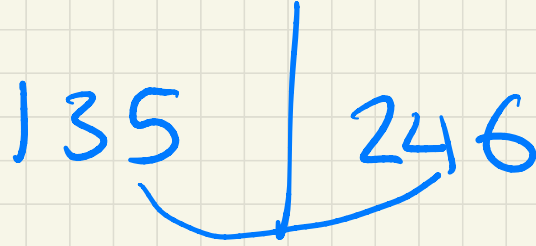


Initial solⁿ 1 2 3 | 4 5 6

Best = 4



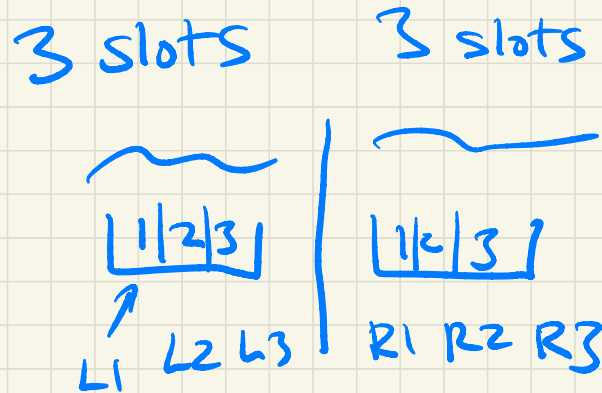
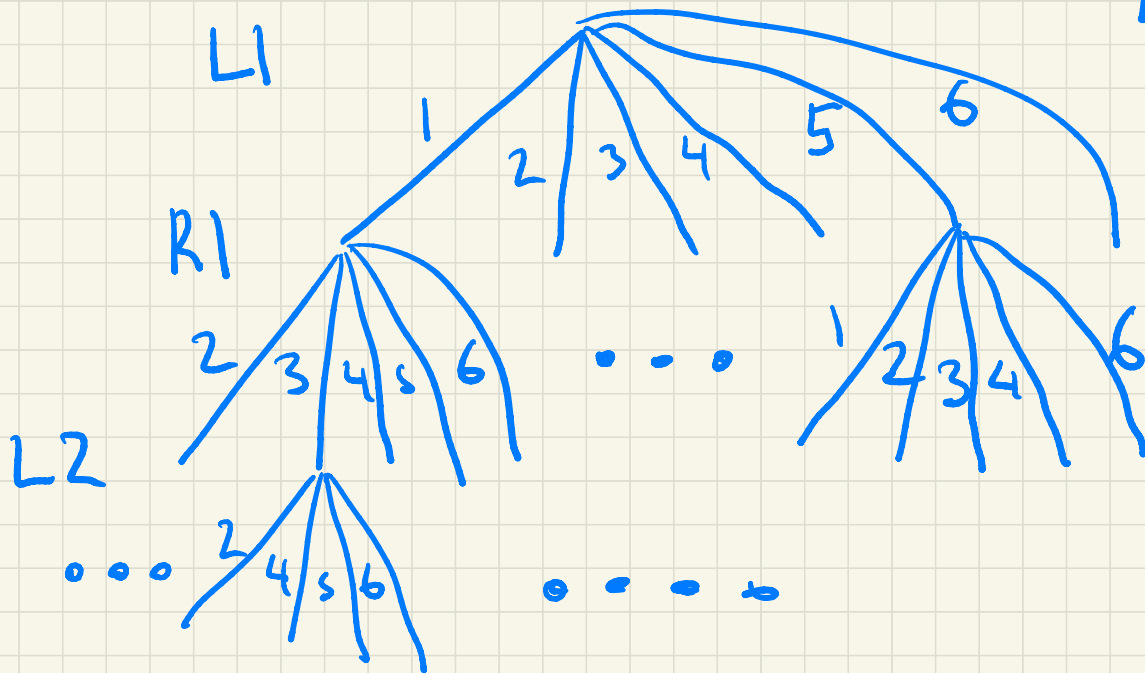
Optimal solution:



crossing count = 1

Design choices

1. Decision tree design



each level
of tree
corresponds
to one
slot

- Good: solves balance problem

- Bad: - much bigger tree
- because this contains
same solution many
times

2. Bounding function (most important)

3. Traversal order

1. BFS

2. DFS

3. lowest bound first



● frontier of built tree

→ build from frontier node with lowest lower bound

⇒ hope it leads to a complete solⁿ cost < Best

4. Initial solution generation
 $\Rightarrow$ as good as possible to
prune more of tree.

Modification: Also compute an upperbound
 $UB(n)$ for interior node n .

if $UB(n) < Best$ replace
 $Best$ (even though its
a partial solution)

- Must be a true upper bound
or will lose optimality

— "Stop before exhaustion" heuristic

— look at $LB(n)$ for all frontier nodes, compare with Best.

— Find maximum difference (max)

— if stop now, we are at most max cost from optimal.

A heuristic-based on $B \approx B$
inexact $B \approx B$ (not A3)

↳ $LB(n)$ is a heuristic to
allow more pruning

→ no longer optimal

→ want $LB(n)$ to higher

A3 example graphics

